

PramaanX

Full Project Context & Technical Reference

AI-Based Fake Identity & Document Screening System

Smart India Hackathon — 4-person team build

Compiled from: P4 Master Prompt · Team Division doc · actual P4 codebase (reviewed and run) · MongoDB

Atlas schema work

Generated September 10, 2026

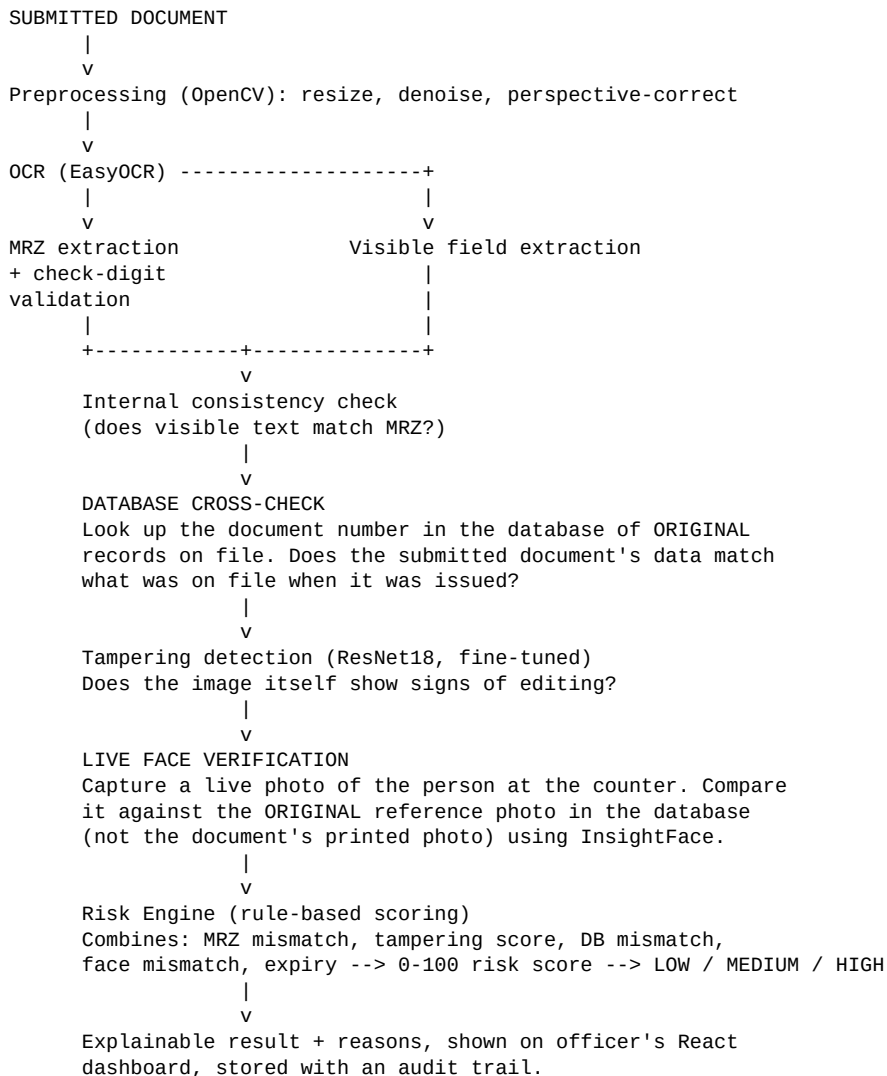
1. What PramaanX Is

PramaanX is an AI-powered document and identity verification platform that analyzes passports, visas, IDs, and other identity documents using OCR, MRZ validation, deep-learning-based tampering detection, face verification against a database of originals, and database cross-checking, to produce an **explainable risk score**. It is decision support for an authorized officer — **not** an automatic fake/genuine verdict.

1.1 What PramaanX explicitly does NOT claim

- Not a legal determination of document authenticity — the officer decides, the system assists.
- Not connected to real government databases — the database is a synthetic/mock reference store for the prototype.
- Not a production-grade liveness/anti-spoofing system — any liveness check built for the hackathon is a basic motion/blink heuristic, not a deep model, and this is stated openly rather than overclaimed.

1.2 End-to-end pipeline



1.3 Key design decision

Face verification compares the live person against the **database's** reference photo, not the document's printed photo. This is stricter than a naive “does the person match their own ID photo” check — it also catches someone presenting a genuine, unaltered document that belongs to someone else.

1.4 Explainability requirement (applies to every module)

Never output a bare score with no reason. Every result should be explainable: not just `is_match: false`, but *why* — similarity too low, no face detected, document not in database, etc. This mirrors the risk engine's own design and is a recurring theme judges will probe in viva.

1.5 Privacy framing (project-wide standing line)

“The prototype uses synthetic/sample identity records and our own team's photos for demonstration. Production deployment would require explicit consent, encrypted storage of biometric templates, a data retention/deletion policy, legal authorization, and compliance with applicable data protection law.”

Don't claim more than what's actually built.

2. Technology Stack

Layer	Technology
Frontend	React.js
Backend	Python + Flask
Database	MongoDB Atlas (mocked as a local JSON store during development)
Image processing	OpenCV
OCR	EasyOCR
MRZ	OCR + MRZ parser + ICAO check-digit validation
Tampering detection	PyTorch + ResNet18 (transfer learning)
Face verification	InsightFace (buffalo_l, pretrained)
Risk scoring	Rule-based (weighted point system)
API communication	REST
Auth	JWT / session-based

3. Team Structure (4-person split)

Person	Role	Owns
P1	Frontend Lead	React UI: Login, Dashboard, Upload, Result screens
P2	Backend Lead	Flask main API, MongoDB Atlas, auth, orchestration, risk engine
P3	AI Engineer — Document	OpenCV preprocessing, EasyOCR, MRZ parsing/validation, ResNet18 tampering model, database cross-check logic
P4	AI Engineer — Biometric & Integration	Live face capture, face verification, liveness, end-to-end integration testing, demo data, pitch deck (Lokesh's role)

This mirrors the natural fault lines in the architecture: **UI** ↔ **API** ↔ **(Document AI + Biometric AI)** ↔ **DB** ↔ **Risk Engine**, so each person can build against a mocked contract before the real thing is ready.

3.1 Division rationale

- **1 frontend + 1 backend** keeps the “plumbing” (UI + API + DB) fully owned so nothing stalls waiting on AI results.
- **2 AI engineers**, split by *document* vs *biometric*, because these are genuinely separate pipelines (OCR/MRZ/tampering vs face matching) with different libraries and no shared code — clean parallel work, no merge conflicts.
- **P4 doubles as integration/QA + pitch owner** since biometric work (using a pretrained model) typically finishes faster than P3's fine-tuning work, freeing up time for the glue work and demo prep that someone needs to own.

3.2 Per-role deliverables

P1 — Frontend (React.js)

- Login page
- Dashboard (stats cards: Documents Screened, High Risk Alerts, Today's Checks)
- Upload screen (document type selector, document file upload, person photo upload)
- Result screen (OCR/MRZ/DB/Face/Tampering status rows, risk score, risk level, reasons list, “Manual Review” button)
- `services/api.js` — a single API layer so backend integration is a one-file swap

Unblock early: build the whole UI against a hardcoded mock JSON (the agreed shape) before P2's API is ready; swap the mock for real fetch calls once `/api/analyze` exists. Stack: React.js, Tailwind/CSS, Axios or fetch.

P2 — Backend (Flask) + Database + Risk Engine

- Flask app skeleton: `/api/login`, `/api/analyze`, `/api/results`
- MongoDB Atlas connection + collections: `users`, `documents`, `verification_results`, `alerts`, `audit_logs`
- JWT/session auth
- Orchestration logic that calls P3's and P4's modules in sequence and assembles the final JSON response
- Risk engine (rule-based): takes OCR/MRZ/tampering/face/DB outputs, computes score, risk level, reasons list
- Seed the documents collection with synthetic reference records for demo

Unblock early: define the internal function signatures for P3 and P4's modules first (see Section 8, Integration Contract) so everyone can build independently. Stack: Python, Flask, PyMongo, PyJWT.

P3 — AI Engineer: Document Analysis

- preprocessing.py — OpenCV (resize, denoise, perspective correction, crop)
- ocr.py — EasyOCR integration -> structured fields (name, DOB, passport no., nationality, expiry)
- mrz.py — MRZ region detection + OCR + MRZ parser + check-digit validation
- validation.py — compare visible fields vs MRZ fields, expiry check, format checks
- tampering.py + train.py — fine-tune ResNet18 (PyTorch) on a genuine-vs-tampered dataset, export tampering_model.pth
- database.py — mock database of original records, with lookup_by_document_number() and a photo_path per record
- db_check.py — compares submitted document's extracted fields against the ORIGINAL record on file
- generate_synthetic_dataset.py — generates synthetic genuine/tampered passport images for training/testing
- webcam_capture.py — captures a live photo from a physical webcam (blind-capture and preview-with-spacebar modes)
- face_match.py — InsightFace-based face embedding + cosine similarity, with verify_against_database()
- main.py — run_full_verification(document_image, live_photo) ties everything together

Priority order if time is short: OCR -> MRZ compare -> tampering model (most time-consuming; start training data prep on Day 1).

Stack: Python, OpenCV, EasyOCR, PyTorch, ResNet18 (ImageNet pretrained).

P4 — AI Engineer: Biometric + Integration/QA + Pitch (this is Lokesh's role)

- face.py — InsightFace/ArcFace integration: face detection -> embeddings -> cosine similarity between document photo and live photo
- Threshold calibration (test similarity scores on the team's own photos to pick a working cutoff)
- Integration testing: once P2's orchestration and P3/P4's modules exist, run end-to-end tests with sample documents and fix glue-code bugs
- Prepare demo dataset (synthetic passports/IDs — genuine and tampered examples) for the live demo
- Own the pitch deck: problem statement, innovation angle (multi-layer evidence fusion + explainability), architecture diagram, live demo script, "what we'd add in production" slide (liveness detection, real DB integration, legal/privacy framing)

Stack: Python, InsightFace/ArcFace (pretrained), PowerPoint/Slides.

3.3 Suggested build order & parallelization (original ~24–36hr hackathon framing)

Phase	P1 (Frontend)	P2 (Backend)	P3 (Document AI)	P4 (Biometric + QA)
Hrs 0-4	Login/Dashboard UI (static)	Flask skeleton + Mongo connection	OpenCV preprocessing + EasyOCR	InsightFace setup + test embeddings
Hrs 4-10	Upload + Result UI (mock data)	/api/analyze route (mocked responses)	MRZ parsing + validation	Face similarity function + threshold testing
Hrs 10-16	Wire UI to real /api/analyze	Real orchestration calling P3/P4 modules	ResNet18 fine-tuning (start early — slowest)	Prep synthetic genuine/tampered demo docs
Hrs 16-22	Polish result screen (reasons, risk styling)	Risk engine + Mongo writes	Finish tampering model integration	End-to-end integration testing, bug fixes
Hrs 22-end	All: polish	All: full end-to-end run-through	All: polish	Pitch deck, demo script, rehearsal

Key rule: start ResNet18 training (P3) as early as possible — it's the slowest-to-iterate piece. Everyone else can build against mocks in the meantime. *Note: this timeboxing was written for a short hackathon sprint — see Section 9 for how it should change now that this is a real multi-week build with teammates integrating continuously.*

4. Integration Contract (agreed Day 1, before splitting up)

P2 defines these function signatures so P3 and P4 build to a fixed interface and can work independently:

```
run_ocr(image_path) -> {"name": str, "dob": str, "passport_number": str,  
                        "nationality": str, "expiry": str}
```

```
run_mrz(image_path) -> {"name": str, "dob": str, "passport_number": str,  
                        "mrz_valid": bool}
```

```
run_tampering(image_path) -> float          # 0.0-1.0 tampering probability
```

```
run_face_match(doc_photo_path, live_photo_path) -> float # 0.0-1.0 similarity
```

```
check_database(passport_number) -> {"found": bool, "expired": bool, "record": dict}
```

The final API response shape (agreed separately) is the contract P1 builds its UI against.

5. P4 Build Brief — Full Detail

P4 builds the **Flask-based biometric verification microservice** — live face capture, face matching against the database's reference photo, optional liveness/anti-spoofing — plus owns **end-to-end integration testing** and the **demo/pitch materials** once the other three parts exist. This is a separate Flask service (not bolted onto P2's main backend) so P3's tampering/OCR work and P4's biometric work don't block each other, and so P2's main API stays clean — it just makes one HTTP call to P4's service and gets a JSON result back.

5.1 Why a separate Flask service

- **Parallel development.** P2 can build `/api/analyze` against a mocked face-verification response before P4's service exists.
- **Clean failure isolation.** If InsightFace crashes or a camera disconnects, it doesn't take down the whole PramaanX backend.
- **Matches how this would actually be deployed.** Biometric services in real identity-verification systems are commonly isolated microservices for security and compliance reasons.

5.2 Functional requirements

5.2.1 Live face capture

- Capture a photo from a webcam at the verification counter.
- Blind capture mode (automated testing) + preview/press-to-capture mode (demo/kiosk).
- Reuse P3's `webcam_capture.py` — don't rebuild it.

5.2.2 Face verification

- Given a live-captured photo and a document number, fetch the reference photo from the database and compute a similarity score.
- Return similarity (0.0-1.0), a match/no-match boolean against a calibrated threshold, and a confidence label.
- Reuse P3's `face_match.py` — `verify_against_database()` already implements this.

5.2.3 Liveness / anti-spoofing (stretch goal, not MVP)

- Detect whether the presented face is a live person vs. a photo, phone screen, or printed image held up to the camera.
- MVP approach: blink/motion check across 2-3 frames rather than a full liveness model.

5.2.4 Integration testing

- Once P2, P3, and P4's pieces all exist, run the full pipeline end-to-end with sample documents + live photos and confirm the JSON response matches the agreed contract.
- Write down what breaks — this is real, valuable work, don't skip it to add more features.

5.2.5 Demo & pitch

- 3-5 test cases for live judging: at least one clean pass, one tampered document, one face mismatch, one document-not-in-database case.

- Pitch deck: problem, innovation (multi-layer evidence fusion + explainability), architecture diagram, live demo, honest “what we’d add for production” slide.

5.3 Flask service design

5.3.1 Folder structure (as briefed)

```
p4_biometric_service/
|
|-- app.py                # Flask app entrypoint
|-- config.py             # thresholds, camera index, etc.
|
|-- routes/
|   |-- capture.py        # /api/capture
|   |-- verify.py         # /api/verify-face
|   |-- health.py         # /api/health
|
|-- services/
|   |-- face_match.py     # copied/imported from P3
|   |-- webcam.py         # copied/imported from P3
|   |-- liveness.py       # new: blink/motion check
|
|-- tests/
|   |-- test_verify.py
|   |-- integration_test.py # full-pipeline test hitting P2 + P3 + P4
|
|-- requirements.txt
|-- README.md
```

5.3.2 Endpoints (as briefed)

GET /api/health — returns {"status": "ok"}. Check this first if something fails live during the demo.

POST /api/capture — triggers a live webcam capture at the counter.

Response:

```
{
  "captured": true,
  "photo_path": "/tmp/captures/live_20260909_1423.jpg"
}
```

POST /api/verify-face — the core endpoint.

Request:

```
{
  "live_photo_path": "/tmp/captures/live_20260909_1423.jpg",
  "document_number": "Q2714253"
}
```

Response:

```
{
  "found_in_database": true,
  "similarity": 0.87,
  "is_match": true,
  "threshold_used": 0.45,
  "liveness_passed": true,
  "confidence_label": "HIGH",
  "error": null
}
```

If the document number isn't in the database, or no face is detected in the live photo, return `found_in_database: false` or `error: "no face detected"` respectively — **never guess a similarity score when a face wasn't actually detected**.

POST /api/liveness-check (stretch) — request: a short sequence of frame paths or a video clip path. Response: {"is_live": true, "reason": "eye blink detected across frames"}

Response contract for P2: keep the response shape stable once P2 starts integrating — changing field names after P2 has wired it up is the #1 source of last-minute breakage.

5.4 Build order (do this in order)

- 1. Skeleton first — /api/health returning {"status": "ok"}, confirm P2 can call it over HTTP.
- 2. Wrap P3's face_match.py in /api/verify-face using a hardcoded test photo pair — no camera yet.
- 3. Wrap P3's webcam_capture.py in /api/capture — test on a real laptop with a camera.
- 4. Calibrate the match threshold using your own team's faces (Section 5.5).
- 5. Liveness check, only after 1-4 work reliably — cut this first if short on time.
- 6. Integration testing — run the full flow through P2's actual API, not just your service alone.
- 7. Demo script + pitch deck last, once you know what actually works live.

5.5 Calibrating the face-match threshold

- Take 5-10 photos of 2-3 different team members under the actual demo lighting (not studio lighting).
- Run compare_faces on every same-person pair and every different-person pair.
- Same-person pairs should cluster noticeably higher than different-person pairs. Pick a threshold between the two clusters — usually ~0.35-0.55 for buffalo_l, but measure, don't assume.
- If the clusters overlap heavily, check lighting/angle before assuming the model is bad.

5.6 Liveness check (MVP version)

- Capture 2-3 frames, ~300ms apart.
- Detect the face + eye landmarks in each frame (InsightFace's landmark output).
- Check for small pixel-level differences around the eye region across frames (a blink, or natural micro-movement).
- A completely static face across all frames -> likely a photo held up to the camera -> fail liveness.
- State clearly in the pitch: this is a basic motion-based liveness check, not a production-grade anti-spoofing model.

5.7 Integration testing checklist

#	Scenario	Expected result
1	Clean genuine document + matching live face	LOW risk, face match
2	Tampered document (P3's synthetic tampered set)	HIGH risk
3	Genuine document + a different team member's face	Face mismatch flagged, risk score reflects it
4	Document number not in the database at all	NOT_FOUND, handled gracefully, no crash
5	No face detected in the live photo (camera pointed at ceiling)	Graceful error, not a 500 crash
6	Flask service down / unreachable	P2's backend degrades gracefully (shows "face verification unavailable", doesn't crash dashboard)

Write down actual results for each, with screenshots if possible — this becomes integration evidence for judges and a sanity check the morning of the demo.

5.8 Viva script — “explain the biometric layer”

“The biometric layer is a separate Flask microservice so it can fail independently without taking down the main system. It uses InsightFace's pretrained buffalo_L model to generate a 512-dimensional face embedding from the live-captured photo, then compares it via cosine similarity against the reference photo stored in the database — not the photo printed on the submitted document, which means we catch an impostor presenting someone else's genuine, unaltered document. We calibrated our match threshold empirically on our own team's photos rather than assuming a textbook value. We also added a lightweight liveness check based on micro-motion across frames to catch the simplest spoofing attempt — holding up a printed photo — while being upfront that it isn't a production-grade anti-spoofing model.”

5.9 requirements.txt (as briefed)

```
Flask>=3.0.0
flask-cors>=4.0.0
opencv-python-headless>=4.8.0
insightface>=0.7.3
onnxruntime>=1.16.0
numpy>=1.24.0
requests>=2.31.0
pytest>=7.4.0
```

6. P4 — Actual Implementation Status (verified by actually running it)

This section covers what has genuinely been built and tested, as of the codebase you uploaded (pramaanx_p4.rar) — reviewed file-by-file and then actually executed with real dependencies, not just read.

6.1 Repository layout as actually built

```
pramaanx_p4/
|-- README.md
|-- pytest.ini
|-- frontend/                # React + Vite operator console (real fetch calls, no fake data)
`-- p4_biometric_service/
    |-- README.md
    |-- app.py                # create_app() factory
    |-- config.py             # Settings dataclass, Settings.from_env()
    |-- requirements.txt
    |-- routes/
    |   |-- capture.py
    |   |-- health.py
    |   |-- index.py
    |   |-- liveness.py
    |   |-- upload.py
    |   |-- verify.py
    |-- services/
    |   |-- database.py
    |   |-- face_match.py     # real InsightFace wrapper, custom exceptions
    |   |-- liveness.py
    |   |-- uploads.py
    |   |-- webcam.py
    |-- scripts/
    |   |-- calibrate_threshold.py
    |-- demo_data/
    |   |-- reference_records.json
    |   |-- photos/           # empty — real team photos must be added locally
    |-- docs/
    |   |-- DEMO_SCRIPT.md
    |   |-- PITCH_DECK.md
    |-- tests/
    |   |-- conftest.py
    |   |-- test_verify.py    # 14 tests, fake-injected matcher
    |-- integration_test.py
```

6.2 Design qualities confirmed by code review

- FaceMatcher never invents a similarity score — unknown documents return similarity: null.
- Distinct exception types: NoFaceDetectedError, ReferenceNotFoundError, ReferencePhotoUnavailableError, ImageInputError, all under a common FaceServiceError base.
- Routes map exceptions to distinct, meaningful HTTP codes: 400 (malformed input), 422 (unusable biometric input — e.g. no face), 503 (dependency unavailable — e.g. missing reference photo).
- Frontend (App.jsx) makes genuine fetch() calls to /api/health, /api/upload-photo, /api/upload-frames, /api/capture, /api/liveness-check, /api/verify-face — no Math.random() or hardcoded demo values anywhere in the UI layer.

6.3 What was actually run in this session (real dependencies, not fakes)

Installed and exercised for real: insightface 2.0, onnxruntime 1.24.4, opencv 4.13.0, numpy 2.4.4. The real **buffalo_I** model (~280MB) was downloaded from InsightFace's GitHub release and loaded successfully.

Real end-to-end result (via Flask test_client(), real FaceMatcher, real DB JSON lookup)

Scenario	Result
Genuine same-identity recapture (mirrored + brightness-shifted real photo)	similarity: 0.930108, is_match: true, confidence_label: HIGH
Unknown document number	similarity: null (never invented), found_in_database: false
No face detected in live photo (test image with no person)	HTTP 422, error: "no face detected", similarity: null

6.4 Two real bugs found by running it, and fixed

Bug 1 — `calibrate_threshold.py` crashed on the first bad photo

The script had no error handling around per-pair face detection. One photo with no detectable face in the input folder crashed the *entire* calibration run with an unhandled traceback — no partial results, no indication of which photo was the problem.

Fix applied: wrapped comparisons in a `_scored_pairs()` helper that catches `FaceServiceError` per pair, logs which pair/reason to stderr, continues with the remaining pairs, and reports a `skipped_pairs` list in the JSON output.

Bug 2 — `found_in_database` was reported incorrectly

`routes/verify.py` hardcoded `found_in_database: false` whenever `NoFaceDetectedError` or `ImageInputError` was raised — even when the document number *had* already been found in the database and the real problem was the live photo. This conflates two very different officer actions: “document doesn't exist” vs “retake the photo.”

Fix applied: `FaceServiceError` now carries a `found_in_database` attribute. `compare_paths()` propagates `found_in_database=True` when reached via `verify_against_database()` (i.e. after the DB record was already confirmed). `routes/verify.py` now reads `exc.found_in_database` instead of hardcoding `False`.

Both fixes were verified against the real model (not fakes), and all 14 existing unit tests still pass — no regressions.

6.5 Genuine remaining gaps (not fixable without the real team)

- `demo_data/photos/` is empty — correctly, for privacy — so nobody has run a real verification with actual reference photos yet.
- The match threshold is still the placeholder `0.45` from the brief. The calibration script is verified working, but the real calibration run (Section 5.5) needs actual team photos under actual demo lighting — that step is on the team, not something that can be done from a sandbox with no camera and no access to real teammates' faces.

7. Database Design — MongoDB Atlas

Collection names match what's already agreed in the team-division document. Field names in `verification_results.face_verification` mirror P4's actual, tested JSON contract (Section 6.3) so P2 can store the response with minimal re-mapping.

7.1 documents — the reference store (what was on file at issuance)

```
{
  "_id": "ObjectId",
  "document_number": "Q2714253",           // unique index
  "document_type": "passport",             // passport | visa | national_id
  "full_name": "string",
  "date_of_birth": "ISODate",
  "nationality": "string",
  "date_of_issue": "ISODate",
  "date_of_expiry": "ISODate",
  "issuing_authority": "string",
  "mrz_string": "string",                   // optional, canonical MRZ as issued
  "reference_photo_url": "string",         // path/URL, NOT raw image bytes
  "created_at": "ISODate",
  "updated_at": "ISODate"
}
```

Index: unique on `document_number`.

7.2 verification_results — one document per submission

```
{
  "_id": "ObjectId",
  "submission_id": "uuid",
  "document_number": "string | null",
  "officer_id": "ObjectId -> users._id",
  "submitted_at": "ISODate",
  "ocr": { "name", "dob", "passport_number", "nationality", "expiry" },
  "mrz": { "name", "dob", "passport_number", "mrz_valid": bool },
  "internal_consistency": { "fields_match": bool, "mismatches": [] },
  "tampering": { "score": 0-1, "is_tampered": bool },
  "database_check": { "found": bool, "expired": bool, "field_mismatches": [] },
  "face_verification": {
    "found_in_database": bool, "similarity": float | null, "is_match": bool,
    "threshold_used": float, "liveness_passed": bool, "liveness_performed": bool,
    "confidence_label": string, "reason": string | null, "error": string | null
  },
  "risk": { "score": 0-100, "level": "LOW | MEDIUM | HIGH", "reasons": [] },
  "manual_review": { "status", "reviewed_by", "reviewed_at", "notes" },
  "created_at": "ISODate"
}
```

Indexes: `document_number`, `submitted_at` (Today's Checks card), `risk.level` (High Risk Alerts card), `manual_review.status`.

7.3 alerts, users, audit_logs, captures

Collection	Purpose	Key indexes
alerts	Flagged MEDIUM/HIGH cases needing officer review; denormalizes <code>risk_level</code> and <code>document_number</code> for fast dashboard queries.	<code>status</code> , <code>created_at</code>
users	Officer/admin accounts. bcrypt <code>password_hash</code> , role enum.	unique email
audit_logs	Append-only action log: login, submit_verification, manual_review_decision, alert_resolved, etc.	timestamp, actor_id

Collection	Purpose	Key indexes
captures (optional)	Live-photo capture metadata with a TTL so files not tied to a kept result auto-expire — privacy hygiene.	TTL on expires_at

7.4 \$jsonSchema validation

Apply Atlas's \$jsonSchema validator to documents and verification_results at creation time, requiring at minimum document_number/document_type/full_name and submitted_at/face_verification/risk respectively. This stops a bug in P2's orchestration from silently writing a half-populated result — enforced at the database layer, not just hoped for in application code.

7.5 Atlas platform features worth using

- **No images inside MongoDB documents.** Atlas M0 (free tier) caps total storage at 512MB, and 16MB is the hard per-document limit on any tier. Store photos on disk or object storage; keep only the path in Mongo.
- **Change streams** on alerts (status: "open") for a live-updating officer dashboard instead of polling.
- **Least-privilege database user** scoped to readWrite on the PramaanX database only — not an Atlas admin-level connection string in .env.
- **Encryption at rest** is on by default on Atlas — worth stating explicitly in the privacy slide.
- **Backups / point-in-time recovery require M10+** — M0 has none. Fine for a prototype, but say so plainly rather than implying coverage that doesn't exist.

7.6 Setting up the real Atlas cluster — steps

Step	Action
1	cloud.mongodb.com -> create a new Project ("PramaanX").
2	Build a Database -> free M0 tier -> region close to where you'll run the demo (e.g. AWS Mumbai).
3	Database Access -> Add New Database User, scoped to readWrite on the PramaanX database only (not your personal Atlas login).
4	Network Access -> Add IP Address for each teammate's network, or temporarily 0.0.0.0/0 during the build (lock down before anything sensitive is in there).
5	Database -> Connect -> Drivers -> Python. Copy the connection string into MONGODB_URI in .env (never hardcoded, never committed).
6	Run create_atlas_collections.py once (Section 7.7) to create all collections with validators and indexes already applied.
7	Add a /api/health check that runs client.admin.command("ping") so a bad connection string or IP-allowlist miss is caught immediately, not mid-demo.

7.7 create_atlas_collections.py

A companion script (delivered separately) creates all six collections idempotently — safe to re-run — with the \$jsonSchema validators above and every index listed in 7.1-7.3, including a TTL index on captures.expires_at. Run it once against the real MONGODB_URI after Step 5 above.

8. Privacy, Ethics & Explainability — Consolidated

8.1 Explainability

Every module's output must carry a reason, not just a verdict. This applies uniformly: OCR/MRZ mismatches list which fields disagree; tampering flags a score, not a bare label; the face layer distinguishes “low similarity” from “no face detected” from “not in database”; the risk engine's final score always ships with a reasons list.

8.2 Privacy

- Only synthetic/sample identity records and the team's own photos are used for the prototype.
- No connection to real government databases.
- Production deployment would require: explicit consent, encrypted storage of biometric templates, a data retention/deletion policy, legal authorization, and compliance with applicable data protection law.
- Live-captured photos should not persist indefinitely — see the optional captures collection with a TTL index (Section 7.3).

8.3 Honesty about limitations

State plainly wherever it applies: the liveness check is a basic motion heuristic, not a production anti-spoofing model; the match threshold is empirically calibrated on a small sample, not a certified accuracy figure; the reference database is synthetic. Judges — and teammates relying on this service — respect stated limitations far more than an overclaimed feature that breaks under scrutiny.

9. Current Status & Outstanding Work

Area	Status	Notes
P4 biometric service — code	Built & unit-tested	14/14 tests pass; real InsightFace pipeline verified end-to-end in this session; 2 real bugs found and fixed.
P4 threshold calibration	Not yet done for real	Script works; needs actual team photos under actual demo lighting — cannot be done remotely.
P4 demo reference photos	Missing (by design)	demo_data/photos/ is empty; must be added locally, not committed to a shared repo without consent.
MongoDB Atlas database	Designed, not yet created	Schema, validators, indexes, and a creation script are ready (Section 7); the live cluster itself has not been provisioned yet.
Full-project integration (Codex)	Reported as merged, unverified	A teammate/tool (“Codex”) has reportedly integrated P1-P4 together. Whether it actually works end-to-end has not yet been verified — the uploaded archive for this review came through empty (0 bytes) and needs to be re-uploaded before a real bug audit and a Codex fix-prompt can be produced.
Liveness check	Stretch goal	Cut first if time is short, per the original brief.
Pitch deck / demo script	Drafted in P4's docs/ folder	Kept intentionally minimal and honest, not over-produced.

9.1 Immediate next actions, in order

- 1. Re-upload the full Codex-integrated project (zip, excluding node_modules/.venv/dist) so a real bug audit can be run the same way the P4 slice was — actually installing dependencies and executing it, not just reading code.
- 2. Stand up the real Atlas cluster using Section 7.6-7.7 and confirm the Flask app's /api/health reports a successful ping.
- 3. Run the real threshold calibration (Section 5.5) with actual team photos under actual demo lighting; update config.py's threshold from the 0.45 placeholder to the measured value.
- 4. Run the full integration testing checklist (Section 5.7) against the real, wired-up system once 1-3 are done.

10. Quick Glossary

Term	Meaning in this project
MRZ	Machine Readable Zone — the two/three lines of OCR-friendly text at the bottom of a passport/ID, with a built-in ICAO check-digit scheme for validation.
Cosine similarity	A 0.0-1.0 measure of how close two face embeddings are; higher means more likely the same person.
buffalo_l	The specific pretrained InsightFace model pack used for face detection + 512-dim embeddings.
Liveness check	A check that the presented face is a live person, not a photo/screen held up to the camera.
Risk engine	Rule-based module that combines every module's output into a 0-100 score and a LOW/MEDIUM/HIGH label with reasons.
found_in_database	Whether the document_number exists in the reference records — distinct from whether the face matched.

End of document. Compiled from the P4 Master Prompt, the team-division document, the actual P4 codebase (reviewed and executed with real dependencies), and the MongoDB Atlas schema work — all produced in this working session.